CYLIXLAB · STUDY GUIDE

Module 04

Computer Vision & Visual Intelligence

Theory | Image Processing · CNN Backbones · ResNet · Object Detection · YOLO · Image Segmentation

What you will master

Section	Topic	You should be able to…
1	Image Processing	Describe how images are stored, convert colour spaces, normalize inputs and apply convolution filters and edge detectors by hand
2	CNN Backbones	Compute output sizes, parameters, FLOPs and receptive fields; trace the evolution from LeNet to EfficientNet and ViT
3	ResNet	Explain the degradation problem, residual learning, basic vs bottleneck blocks and ResNet depth counting
4	Object Detection	Convert box formats, compute IoU, run NMS, compare two-stage and one-stage detectors and read mAP
5	YOLO	Derive the output tensor, decode predicted boxes, explain the loss and the version history
6	Image Segmentation	Distinguish semantic, instance and panoptic tasks; explain FCN, U-Net, DeepLab and Mask R-CNN; compute IoU and Dice
7	Revision Kit	Choose a model for a task, use the formula sheet and avoid classic mistakes

KEY IDEA

The big idea: an image is a grid of numbers. **Filters** turn pixels into features, and a **CNN backbone** stacks learned filters into a hierarchy: edges → textures → parts → objects. Put a classifier on the backbone and you get **image classification**; add heads that regress boxes and you get **object detection** (YOLO, Faster R-CNN); add an upsampling decoder and you get per-pixel **segmentation** (U-Net, Mask R-CNN). **ResNet**'s residual connections are what made deep backbones trainable.

Era	Approach	Limitation that motivated the next step
Before 2012	Hand-crafted features (SIFT, HOG) + SVM; sliding windows	Features designed by hand; brittle and slow for detection
2012	AlexNet: deep CNN trained on GPUs wins ImageNet	Deeper plain networks became hard to train
2014–2015	VGG, GoogLeNet; R-CNN → Fast / Faster R-CNN; FCN, U-Net	Very deep nets degraded; two-stage detection was slow
2015–2016	ResNet (residual learning); YOLO and SSD (real-time one-stage)	Small objects, accuracy vs speed trade-off
2017 →	FPN, RetinaNet, Mask R-CNN; EfficientNet; Vision Transformers (ViT, DETR); modern YOLO	Efficiency, edge deployment, open-vocabulary and foundation models

1 Image Processing Fundamentals

A digital image is an array of numbers. A grayscale image has shape **H × W**; a colour image has shape **H × W × C** with C = 3 channels. With 8-bit storage each value is an integer in **0–255**. Everything a vision model does, from blurring to detecting a car, is arithmetic on this array.

Colour space	Channels	What they mean	Typical use
Grayscale	1	Intensity (brightness)	Documents, X-rays, edge detection
RGB	3	Red, green, blue light	Default input for CNNs
BGR	3	Same as RGB, reversed order	OpenCV's default when loading images
HSV	3	Hue, saturation, value	Colour-based masking, robust to lighting changes
YCbCr / Lab	3	Brightness separated from colour	Compression (JPEG), perceptual colour distances

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

WORKED EXAMPLE
Pixel (R, G, B) = (200, 100, 50) → Y = 59.8 + 58.7 + 5.7 = **124.2 ≈ 124**. Green gets the largest weight because the eye is most sensitive to it.
Memory: a 1920 × 1080 RGB image = 1920·1080·3 = **6,220,800 bytes ≈ 6.2 MB** as uint8, and 4× that (≈ 24.9 MB) once converted to float32 for a network.

1.1 Point operations and normalization

Point operations change each pixel independently: brightness (add), contrast (multiply), gamma correction (x^{γ}) and **histogram equalization**, which spreads intensities so that a dark, low-contrast image uses the full range. Before a network sees an image, pixels are scaled to [0, 1] and then **standardized** per channel:

$$x' = \frac{x - \mu_c}{\sigma_c}$$

- ImageNet-pretrained models expect μ = (0.485, 0.456, 0.406) and σ = (0.229, 0.224, 0.225) for (R, G, B). Inference must use **exactly** the same preprocessing as training.
- Tensor layout differs between frameworks: PyTorch uses **N × C × H × W** (channels-first), TensorFlow/Keras defaults to **N × H × W × C** (channels-last).

1.2 Convolution and filters

A **filter (kernel)** is a small matrix slid across the image. At every position the overlapping values are multiplied element-wise and summed. Deep-learning libraries actually compute cross-correlation (no kernel flip), but the operation is still called convolution.

$$(I * K)(i,j) = \sum_{m}\sum_{n} I(i+m,\ j+n)\,K(m,n)$$

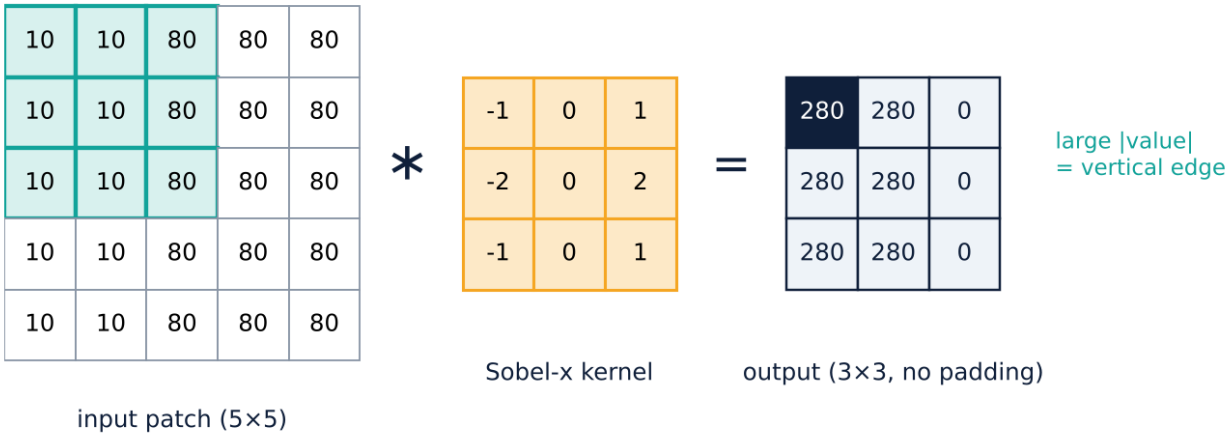


Figure 1: A 3×3 Sobel-x kernel slid over a step edge. Left-to-right intensity jumps give a large response; flat regions give 0.

WORKED EXAMPLE
Top-left window rows are [10, 10, 80]. Sobel-x weights each row as −1·left + 1·right, and the middle row counts twice: (−10 + 80)·1 + (−10 + 80)·2 + (−10 + 80)·1 = 70 + 140 + 70 = **280**.
The right-most window lies entirely in the 80 region: (−80 + 80)·(1 + 2 + 1) = **0**. Edges are where neighbouring pixels differ, and the kernel measures exactly that difference.

Filter	Kernel idea	Effect	Use
Box (mean)	All weights 1/9	Blurs; averages noise	Quick smoothing
Gaussian	Bell-shaped weights, e.g. [1 2 1; 2 4 2; 1 2 1]/16	Smooth blur, fewer artefacts	Denoise before edge detection
Median	Non-linear: middle value of the window	Removes salt-and-pepper noise, keeps edges	Impulse noise
Sobel x / y	[−1 0 1; −2 0 2; −1 0 1] and its transpose	First derivative (gradient)	Edges and their direction
Laplacian	[0 1 0; 1 −4 1; 0 1 0]	Second derivative	Fine edges; very noise-sensitive
Sharpen	Identity + scaled Laplacian	Boosts edges	Image enhancement

1.3 Edges, thresholds and morphology

$$G = \sqrt{G_x^2 + G_y^2}, \qquad \theta = \arctan(G_y / G_x)$$

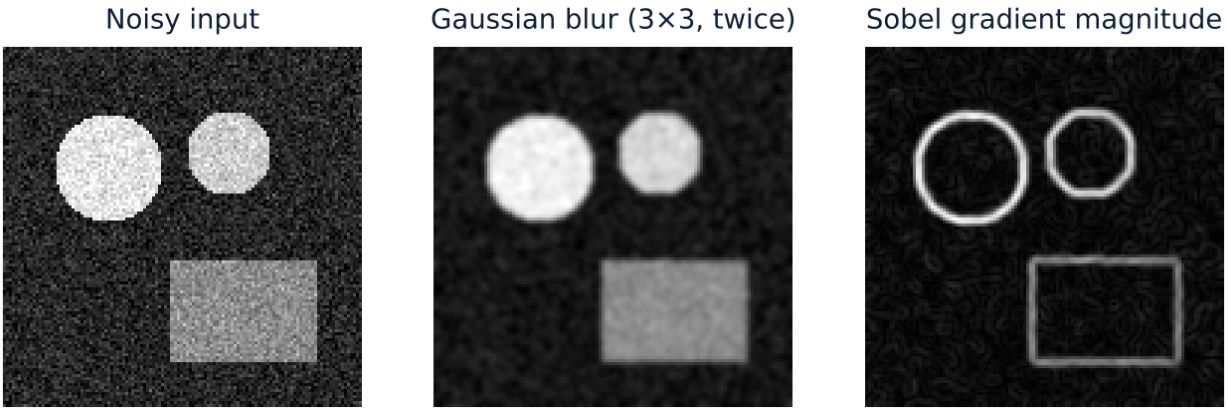


Figure 2: Smoothing first, then Sobel: noise is suppressed and the object boundaries stand out.

- **Canny edge detector:** Gaussian smoothing → gradient magnitude and direction → **non-maximum suppression** (thin edges to 1 pixel) → double threshold with **hysteresis** (weak edges kept only if connected to strong ones).
- **Thresholding:** pixel > T → foreground. **Otsu's method** chooses T automatically by minimizing the variance within the two classes.
- **Morphology** on binary masks: **erosion** shrinks objects, **dilation** grows them, **opening** (erode then dilate) removes small specks, **closing** (dilate then erode) fills small holes.

1.4 Data augmentation

Augmentation creates new training samples by transforming existing ones, teaching the model which changes should **not** change the label. It is one of the cheapest ways to reduce overfitting in vision.

Augmentation	Teaches invariance to	Caution
Horizontal flip	Left–right orientation	Not for text or digits where direction matters
Random crop / scale	Position and size	Must move boxes and masks with the image
Rotation, affine	Camera angle	Box corners change; recompute the box
Colour jitter	Lighting and white balance	Avoid when colour is the label (e.g. ripe fruit)
Cutout / Mixup / CutMix	Occlusion; sharper decision boundaries	Labels become mixed for Mixup/CutMix
Mosaic (4 images in one)	Context and small objects	Popular in YOLO training; turn off near the end

COMMON PITFALL
Preprocessing mismatch. Feeding BGR images (OpenCV) to a model trained on RGB, skipping normalization, or resizing differently at inference silently lowers accuracy without any error message. For detection and segmentation, any geometric augmentation must be applied to the **labels** (boxes, masks) too.

2 CNN Backbones

A **backbone** is the feature extractor: a stack of convolution, normalization, activation and pooling layers that turns an image into feature maps. A **head** uses those features for a task (classify, detect, segment). CNNs suit images because of **local connectivity** (pixels relate most to their neighbours), **weight sharing** (the same detector works everywhere) and **translation equivariance** (shift the input, the feature map shifts too).

2.1 Output size

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

WORKED EXAMPLE
ResNet stem: 224 × 224 input, 7 × 7 conv, stride 2, padding 3 → ⌊(224 − 7 + 6)/2⌋ + 1 = ⌊111.5⌋ + 1 = **112**. Then 3 × 3 max-pool, stride 2, padding 1 → ⌊(112 − 3 + 2)/2⌋ + 1 = ⌊55.5⌋ + 1 = **56**. Two stride-2 steps have already reduced the resolution 4×.

2.2 Parameters and compute

$$\text{params} = (K \cdot K \cdot C_{in} + 1) \cdot C_{out}, \qquad \text{MACs} = K^2 C_{in} C_{out} H_{out} W_{out}$$

WORKED EXAMPLE
3 × 3 conv, 64 → 128 channels: (9·64 + 1)·128 = **73,856** parameters, independent of image size.
On a 56 × 56 output it performs 9·64·128·3,136 = **231,211,008 ≈ 0.23 G** multiply-accumulates. Parameters set *memory*; spatial size sets *compute*, which is why early high-resolution layers are expensive.

2.3 Receptive field

The **receptive field** is the region of the input that can influence one output value. Stacking small kernels grows it: n stacked 3 × 3 convolutions (stride 1) see a **(2n + 1) × (2n + 1)** region. VGG used this to replace large kernels with stacks of 3 × 3.

Option	Receptive field	Weights (C → C channels)	Non-linearities
One 5 × 5 conv	5 × 5	$25C^2$	1
Two 3 × 3 convs	5 × 5	$18C^2$ (28% fewer)	2
One 7 × 7 conv	7 × 7	$49C^2$	1
Three 3 × 3 convs	7 × 7	$27C^2$ (45% fewer)	3

Dilated (atrous) convolution spaces the kernel taps d pixels apart, enlarging the receptive field with no extra parameters: $K_{eff} = K + (K - 1)(d - 1)$, so a 3 × 3 kernel with d = 2 covers 5 × 5.

2.4 Evolution of backbones

Model	Year	Params	Key idea
LeNet-5	1998	≈ 60K	Conv → pool → FC for digit recognition
AlexNet	2012	≈ 60M	ReLU, dropout, GPU training; ImageNet breakthrough
VGG-16	2014	≈ 138M	Uniform stacks of 3 × 3 convs; simple but heavy
GoogLeNet	2014	≈ 7M	Inception modules; 1 × 1 convs to cut channels
ResNet-50	2015	≈ 25.6M	Residual connections; trainable at 50–152+ layers
MobileNetV2	2018	≈ 3.4M	Depthwise-separable convs, inverted residuals; mobile
EfficientNet-B0	2019	≈ 5.3M	Compound scaling of depth, width and resolution
ViT-B/16	2020	≈ 86M	Image as 16 × 16 patches fed to a Transformer

2.5 Efficient convolutions and pooling

- **Depthwise-separable convolution** = one K × K filter per channel (depthwise) + a 1 × 1 conv to mix channels (pointwise). Weights: $K^2 \cdot C_{in} + C_{in} \cdot C_{out}$ instead of $K^2 \cdot C_{in} \cdot C_{out}$. For 3 × 3, 128 → 128: 1,152 + 16,384 = 17,536 vs 147,456, about **8.4× fewer**.
- **1 × 1 convolution** mixes channels at each pixel; used to shrink or expand depth cheaply.
- **Max pooling** keeps the strongest response in each window (no parameters) and adds small translation invariance.
- **Global average pooling (GAP)** averages each channel to one number: 7 × 7 × 2048 → 2048. An FC layer to 1,000 classes then needs 2,049,000 parameters, versus 100,353,000 if the 100,352 values were flattened.

2.6 Transfer learning

Strategy	What is trained	When to use
Feature extraction	Only the new head; backbone frozen	Small dataset similar to ImageNet
Partial fine-tuning	Head + last stages	Medium dataset or moderate domain shift
Full fine-tuning	Everything, with a small learning rate	Large dataset or very different domain (medical, satellite)

3 ResNet: Residual Networks

In 2015 He et al. observed the **degradation problem**: a 56-layer plain CNN had **higher training error** than a 20-layer one. This is not overfitting (training error is worse too); the optimizer simply struggles to make extra layers useful. A deeper net could in theory copy the shallow one and set extra layers to identity, but plain layers find identity hard to learn.

Residual learning changes what each block learns. Instead of the full mapping H(x), the block learns the residual F(x) = H(x) − x and adds the input back through a **shortcut**:

$$y = F(x, \{W_i\}) + x, \qquad \frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + I$$

- **Identity is easy:** if a block is not needed, the weights only have to push F(x) toward 0.
- **Gradient highway:** the "+ I" term lets gradients reach early layers directly, countering vanishing gradients.
- **No extra parameters** for an identity shortcut; the addition is element-wise.

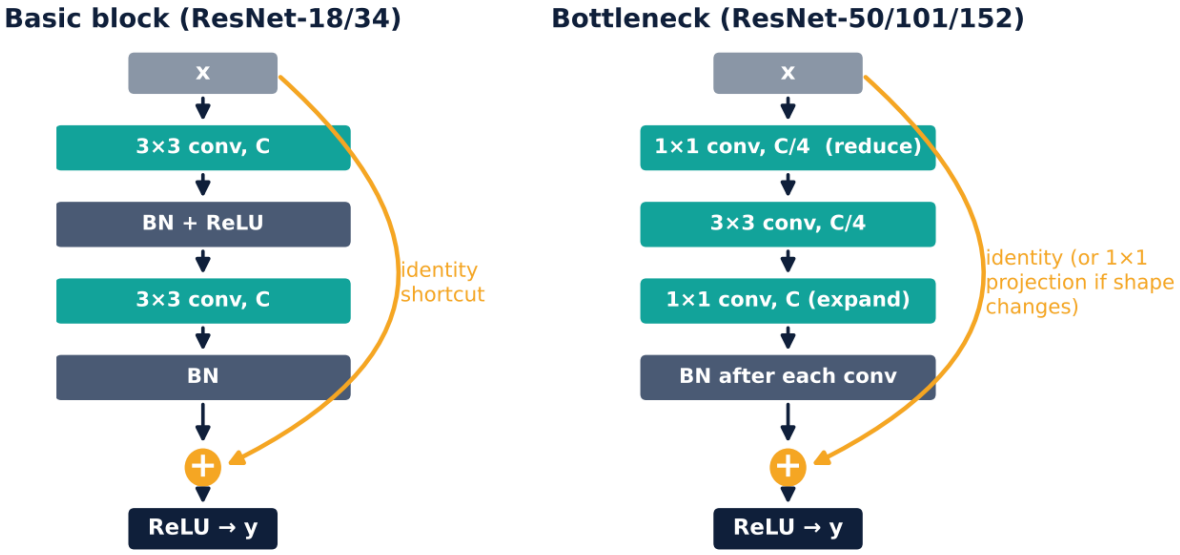


Figure 3: Basic block (two 3×3 convs) and bottleneck block (1×1 reduce, 3×3, 1×1 expand). BN follows every conv.

3.1 Projection shortcuts

The addition needs F(x) and x to have the same shape. When a stage starts with stride 2 or changes the channel count, the shortcut becomes a **1 × 1 convolution with stride 2** (plus BN) that matches the new shape. All other shortcuts stay as free identities.

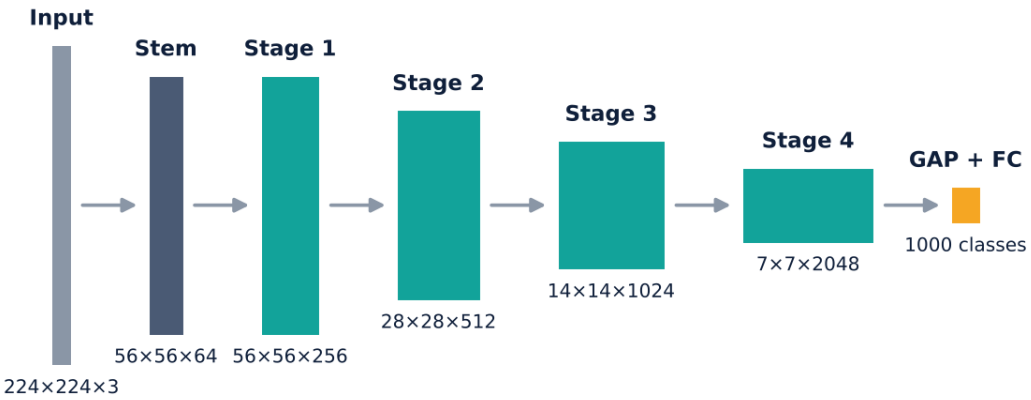
3.2 The bottleneck block

WORKED EXAMPLE
Block with 256 input/output channels. Two plain 3 × 3 convs at 256 channels: $2 \cdot 9 \cdot 256^2$ = **1,179,648** weights. Bottleneck: 1 × 1 (256 → 64) = 16,384; 3 × 3 (64 → 64) = 36,864; 1 × 1 (64 → 256) = 16,384; total **69,632**, about **17× fewer**. This is what makes ResNet-50 and deeper practical.

3.3 The ResNet family

Model	Block	Blocks per stage	Params	GFLOPs (224^2)
ResNet-18	Basic	[2, 2, 2, 2]	11.7M	1.8
ResNet-34	Basic	[3, 4, 6, 3]	21.8M	3.6
ResNet-50	Bottleneck	[3, 4, 6, 3]	25.6M	4.1
ResNet-101	Bottleneck	[3, 4, 23, 3]	44.5M	7.8
ResNet-152	Bottleneck	[3, 8, 36, 3]	60.2M	11.5

WORKED EXAMPLE
Counting layers with weights: ResNet-50 = 1 stem conv + 16 bottleneck blocks × 3 convs + 1 FC = 1 + 48 + 1 = **50**. ResNet-34 = 1 + 16 basic blocks × 2 + 1 = **34**. Shortcut projections and BN are not counted.



Spatial size shrinks (÷32 overall) while channels grow: early layers see edges, deep layers see parts and objects

Figure 4: ResNet-50 feature hierarchy. Each stage halves the resolution and increases channels; GAP + FC classify.

- **Successors:** ResNeXt (grouped convolutions, “cardinality”), Wide ResNet (wider, shallower), DenseNet (concatenates all previous features), ConvNeXt (2022, a ResNet modernized with Transformer-era design choices).
- **Default backbone:** ResNet-50/101 with an FPN is the classic backbone for Faster R-CNN, Mask R-CNN and DeepLab.
- **Order inside a block:** conv → BN → ReLU, with the final ReLU applied *after* the addition.

COMMON PITFALL
BatchNorm modes. BN uses batch statistics in training and stored running averages at inference. Forgetting to switch to evaluation mode (e.g. *model.eval()*) makes predictions depend on the batch and can wreck accuracy.

4 Object Detection

Detection answers two questions at once: **what** objects are present and **where** each one is, as a bounding box with a class label and a confidence score. The number of objects varies from image to image.

Task	Output	Example
Classification	One label for the whole image	“cat”
Localization	One label + one box	“cat” at (40, 30, 200, 180)
Object detection	Variable number of boxes, labels and scores	3 people, 1 bicycle
Semantic segmentation	Class for every pixel	All person pixels = 1

Task	Output	Example
Instance segmentation	Mask for every object	Person #1, person #2 separately

4.1 Bounding-box formats

Format	Values	Used by
Corners	(x_1, y_1, x_2, y_2) in pixels	Pascal VOC, torchvision
Top-left + size	(x, y, w, h) in pixels	COCO annotations
Centre + size, normalized	(c_x, c_y, w, h) divided by image width/height, in [0, 1]	YOLO label files

WORKED EXAMPLE
Centre format $(c_x, c_y, w, h) = (50, 40, 20, 10) \rightarrow x_1 = 50 - 10 = 40$, $y_1 = 40 - 5 = 35$, $x_2 = 60$, $y_2 = 45$, i.e. **(40, 35, 60, 45)**. In a 100 × 80 image the YOLO label is (0.50, 0.50, 0.20, 0.125).

4.2 Intersection over Union (IoU)

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

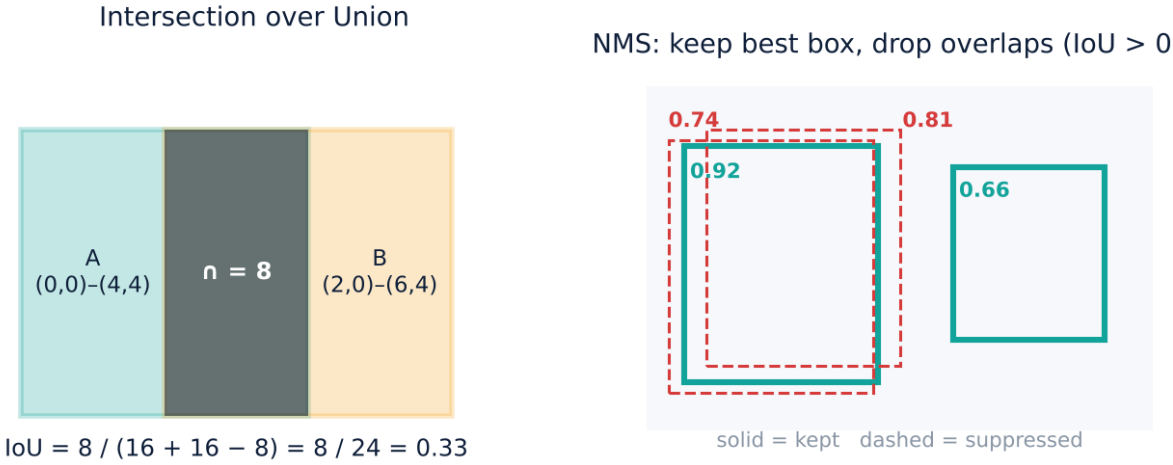


Figure 5: Left, IoU of two 4×4 boxes overlapping by half. Right, NMS keeps one box per object.

WORKED EXAMPLE
A = (0, 0, 4, 4), B = (2, 2, 6, 6). Intersection = (2, 2, 4, 4) → 2 × 2 = 4. Union = 16 + 16 − 4 = 28. IoU = 4 / 28 = **0.143**. Shifting B to overlap half of A (Figure 5) gives 8/24 = 0.33; identical boxes give 1.

4.3 Non-Maximum Suppression (NMS)

A detector usually fires several overlapping boxes for one object. NMS keeps the best and removes duplicates, separately for each class:

- 1. Sort boxes by confidence score.
- 2. Take the highest-scoring box and keep it.
- 3. Remove every remaining box whose IoU with it exceeds a threshold (typically 0.45–0.5).
- 4. Repeat with the remaining boxes until none are left.

WORKED EXAMPLE
Scores 0.92, 0.81, 0.74 on one person and 0.66 on another. Keep 0.92; its IoU with 0.81 is 0.71 and with 0.74 is 0.84, both > 0.5, so both are removed. The 0.66 box has IoU ≈ 0 with the kept box and survives. Result: **two** detections. A threshold that is too low can delete a real second object in a crowd; Soft-NMS lowers scores instead of deleting.

4.4 Two-stage vs one-stage detectors

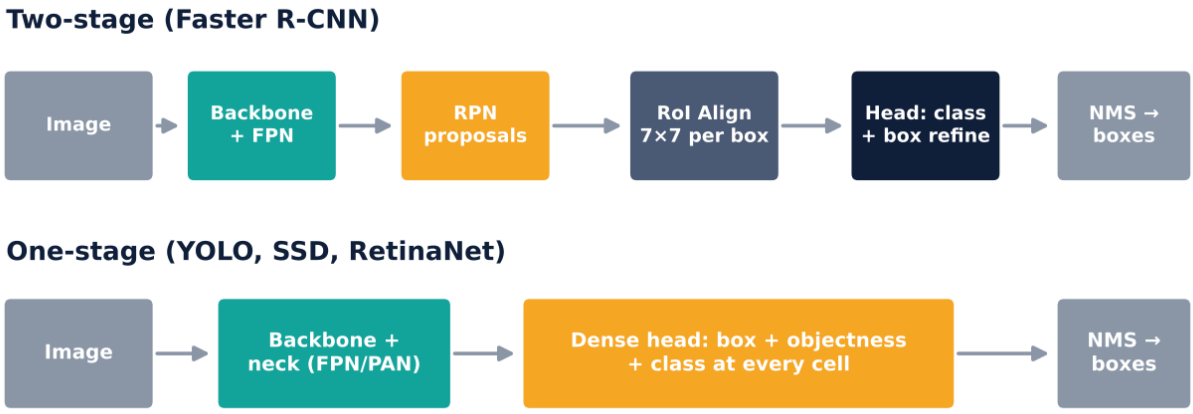


Figure 6: Two-stage detectors propose regions then classify them; one-stage detectors predict densely in one pass.

Model	Year	Type	Key idea
R-CNN	2014	Two-stage	~2,000 selective-search regions, a CNN run on each; very slow
Fast R-CNN	2015	Two-stage	One shared feature map; RoI pooling per region
Faster R-CNN	2015	Two-stage	Region Proposal Network (RPN) with anchors replaces selective search
YOLO	2016	One-stage	Grid-based regression of boxes and classes; real-time
SSD	2016	One-stage	Default boxes on several feature-map scales
RetinaNet	2017	One-stage	FPN + focal loss closes the accuracy gap to two-stage
DETR	2020	Transformer	Set prediction with bipartite matching; no anchors or NMS

4.5 Anchors, FPN and focal loss

- **Anchors** are predefined box shapes (scales × aspect ratios) at each feature-map location. The network predicts *offsets* from an anchor rather than raw coordinates, which is easier to learn.
- **Feature Pyramid Network (FPN):** a top-down path with lateral connections gives every resolution strong semantics. Small objects are predicted on high-resolution levels, large ones on coarse levels.
- **Class imbalance:** a dense detector evaluates ~100k locations, almost all background. Easy negatives swamp the loss.

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log p_t$$

WORKED EXAMPLE
With $\gamma = 2$, an easy example with $p_t = 0.9$ is scaled by $(1 - 0.9)^2 = \mathbf{0.01}$, i.e. 100× down-weighted. A hard example with $p_t = 0.1$ keeps $(0.9)^2 = 0.81$ of its loss, so training focuses on hard cases.

4.6 Evaluation: precision, recall and mAP

A prediction is a **true positive** if it matches a not-yet-matched ground-truth box of the same class with IoU ≥ threshold; otherwise it is a false positive. Unmatched ground-truth boxes are false negatives.

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad AP = \int_0^1 p(r)\,dr, \qquad mAP = \frac{1}{N}\sum_{c=1}^{N} AP_c$$

WORKED EXAMPLE
7 correct detections, 3 false alarms, 2 missed objects: P = 7/10 = **0.70**, R = 7/9 = **0.78**. Sweeping the confidence threshold traces a precision–recall curve; AP is the area under it, and mAP averages AP over classes.

- **mAP@0.5** (Pascal VOC style) counts a match at IoU ≥ 0.5.
- **COCO mAP@[.5:.95]** averages mAP over 10 IoU thresholds (0.50, 0.55, …, 0.95), rewarding precise localization; it is always lower than mAP@0.5 for the same model.

COMMON PITFALL
Comparing a model's mAP@0.5 with another model's COCO mAP@[.5:.95] is meaningless; check which metric, which dataset split and which input resolution were used before comparing numbers.

5 YOLO: You Only Look Once

YOLO (Redmon et al., 2016) treats detection as a **single regression problem**: one forward pass maps the image straight to boxes and class probabilities. The original ran at about 45 frames per second, which made real-time detection practical. YOLO sees the whole image at once, so it makes fewer background false positives than sliding-window methods.

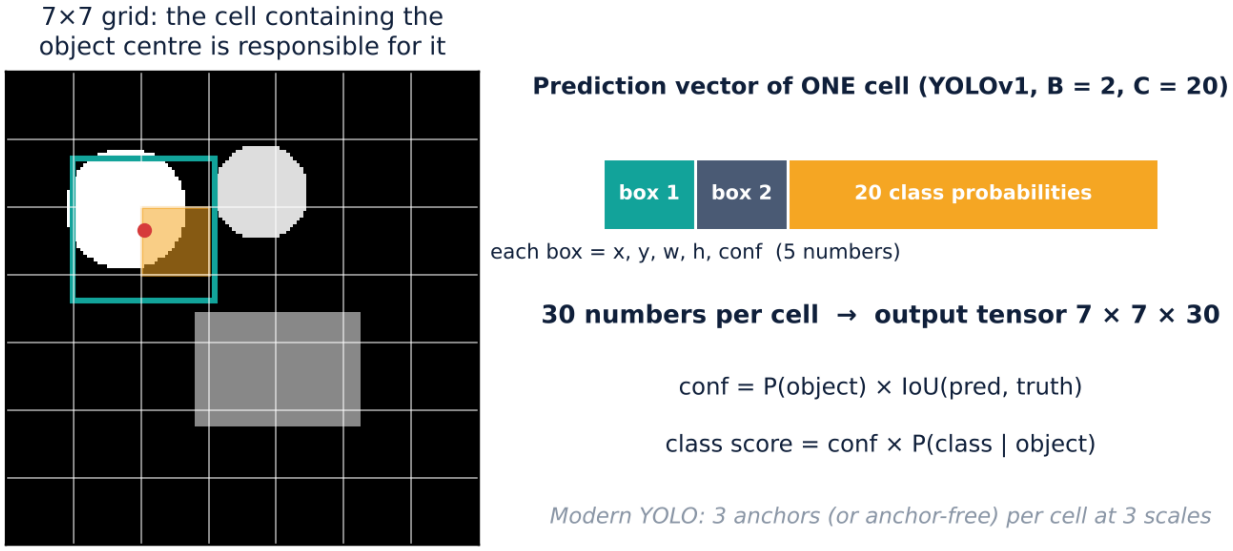


Figure 7: The image is divided into an S×S grid; each cell predicts B boxes and C class probabilities.

5.1 The output tensor

$$\text{output} = S \times S \times (B \cdot 5 + C)$$

WORKED EXAMPLE
YOLOv1 on Pascal VOC: S = 7, B = 2, C = 20 → 7 × 7 × (2·5 + 20) = 7 × 7 × 30 = **1,470** numbers.
At most 7·7·2 = **98** boxes per image, and each cell predicts only **one** set of class probabilities. So two small objects of different classes whose centres fall in the same cell cannot both be detected: v1's main weakness.

5.2 Box parameterization (YOLOv2 onwards)

Later versions predict offsets t relative to the cell's top-left corner (c_x, c_y) and an anchor's size (p_w, p_h). The sigmoid keeps the centre inside its cell, and the exponential keeps width and height positive:

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = p_w e^{t_w}, \quad b_h = p_h e^{t_h}$$

WORKED EXAMPLE
416 × 416 input, stride 32 → 13 × 13 grid. Cell (c_x, c_y) = (3, 4), $t_x = t_y = 0$, anchor $p_w = 2$ cells, $t_w = \ln 1.5 \approx 0.405$.
$b_x = 0.5 + 3 = 3.5$ cells → 3.5 × 32 = **112 px**; $b_y = 4.5$ cells → **144 px**; $b_w = 2 \times 1.5 = 3$ cells → **96 px**.

5.3 Multi-scale prediction

YOLOv3 predicts at three scales with strides 32, 16 and 8, so large objects are handled on the coarse map and small ones on the fine map, much like an FPN.

WORKED EXAMPLE
At 416 × 416 the grids are 13 × 13, 26 × 26 and 52 × 52, each cell with 3 anchors: 3 × (169 + 676 + 2,704) = **10,647** candidate boxes. For COCO's 80 classes each scale has 3 × (5 + 80) = **255** output channels.

5.4 The loss

Term	YOLOv1	Modern YOLO (v5/v8 style)
Box	Squared error on x, y, √w, √h with $\lambda_{coord} = 5$	IoU-based loss (CIoU), plus distribution focal loss in v8
Objectness	Squared error on confidence; $\lambda_{noobj} = 0.5$ for empty cells	Binary cross-entropy (v8 folds it into class scores)
Class	Squared error on class probabilities	Binary cross-entropy per class (allows overlapping labels)

- The square root on w and h makes a 5-pixel error count more for a small box than for a large one.
- $\lambda_{noobj} = 0.5$ down-weights the many empty cells, the same imbalance that focal loss attacks.

5.5 Version history

Version	Year	Main contributions
YOLOv1	2016	Single-pass grid regression; real-time detection
YOLOv2 / YOLO9000	2017	Anchors from k-means, batch norm, Darknet-19, multi-scale training
YOLOv3	2018	Darknet-53 with residual blocks, 3-scale predictions, BCE class loss
YOLOv4	2020	CSPDarknet53, PANet neck, Mosaic augmentation, CIoU loss
YOLOv5	2020	PyTorch implementation by Ultralytics; easy training and export; n/s/m/l/x sizes
YOLOv7	2022	E-ELAN blocks and “trainable bag-of-freebies”
YOLOv8	2023	Anchor-free, decoupled head; same framework for detection, segmentation and pose
YOLOv10	2024	NMS-free training with consistent dual assignments, lowering latency

5.6 Inference pipeline

- **Letterbox** resize: scale the image to fit (e.g. 640 × 640) while keeping aspect ratio, pad the rest.
- Forward pass → filter boxes below a **confidence threshold** (often 0.25) → **NMS** (IoU ≈ 0.45–0.7).
- Undo the letterbox scaling and padding to map boxes back to original image coordinates.
- Model size trades speed for accuracy: nano/small for edge devices, large/x-large for maximum mAP.

COMMON PITFALL
Forgetting to undo the letterbox padding shifts every box; a very high confidence threshold hides real objects (low recall), while a very low one floods the output with false positives.

6 Image Segmentation

Segmentation assigns a label to **every pixel**. It gives exact shapes rather than boxes and is used in medical imaging, autonomous driving, satellite mapping and photo editing.

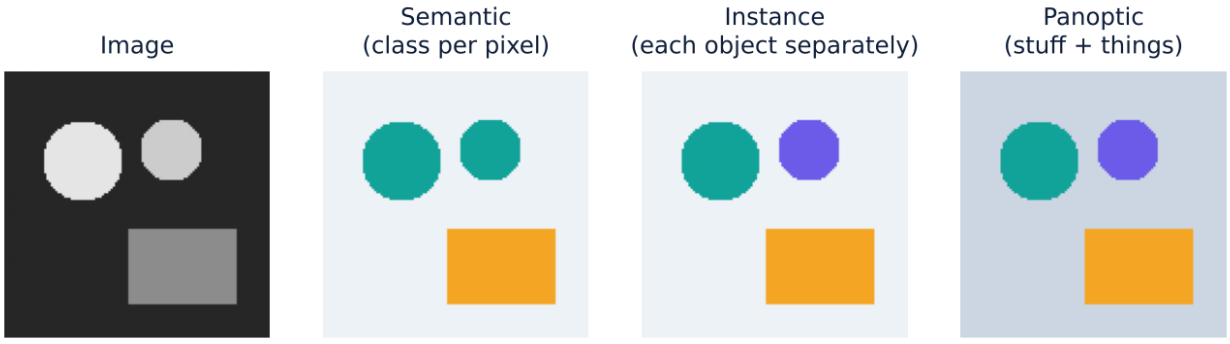


Figure 8: Semantic segmentation merges both circles into one class; instance segmentation separates them; panoptic also labels background “stuff”.

Task	Labels	Distinguishes objects of the same class?	Typical models
Semantic	Class per pixel	No	FCN, U-Net, DeepLab
Instance	Mask per countable object (“things”)	Yes	Mask R-CNN, YOLOv8-seg
Panoptic	Class for every pixel + instance ID for things	Yes, and covers “stuff” (sky, road)	Panoptic FPN, Mask2Former

6.1 Fully Convolutional Networks (FCN)

FCN (2015) replaced a classifier's fully-connected layers with convolutions, so the network outputs a coarse **class score map** for any input size. The map is upsampled back to H × W, and skip connections from earlier, higher-resolution layers (FCN-16s, FCN-8s) sharpen the boundaries. Training uses per-pixel softmax cross-entropy.

6.2 Upsampling

Nearest-neighbour and bilinear interpolation have no parameters. A **transposed convolution** learns how to upsample:

$$O = (I - 1)\,S - 2P + K$$

WORKED EXAMPLE
I = 16, stride S = 2, padding P = 1, kernel K = 4 → (16 − 1)·2 − 2 + 4 = **32**: exact 2× upsampling. Kernels that overlap unevenly cause **checkerboard artefacts**; resize-then-conv is a common fix.

6.3 U-Net

U-Net (Ronneberger et al., 2015), designed for biomedical images, has a contracting **encoder** that captures context and an expanding **decoder** that recovers resolution. **Skip connections** concatenate encoder features with decoder features at the same scale, so fine boundaries lost by pooling are restored. With heavy augmentation it trains well from only tens of labelled images.

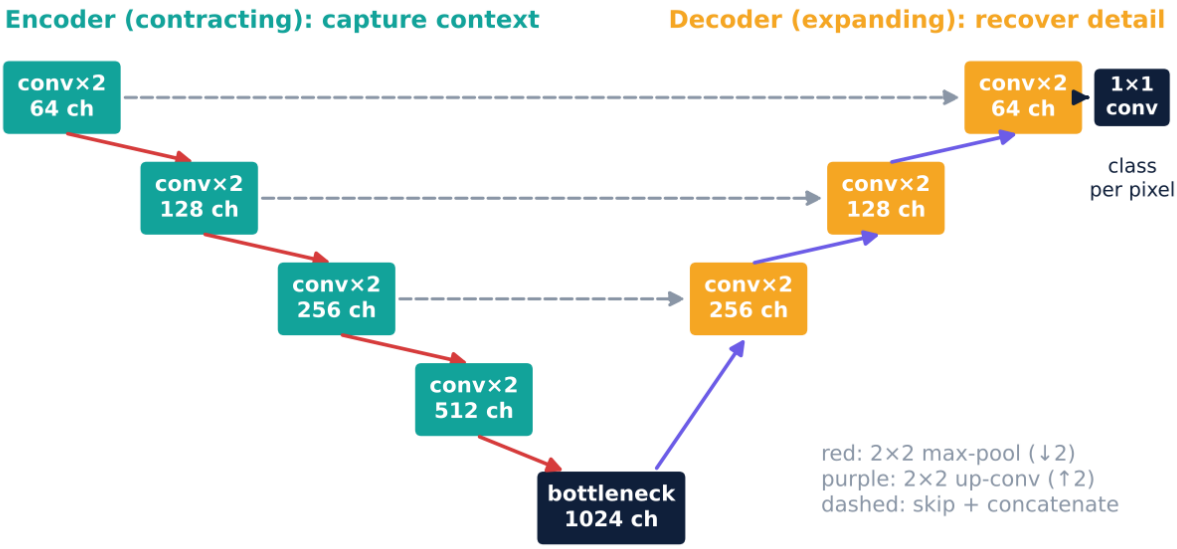


Figure 9: U-Net. Pooling halves resolution in the encoder; up-convolutions double it in the decoder; skips carry detail across.

6.4 DeepLab and atrous convolution

DeepLab keeps feature maps at higher resolution by using **dilated (atrous) convolutions** instead of extra downsampling, and **ASPP** (atrous spatial pyramid pooling) runs parallel dilations (e.g. 6, 12, 18) to capture objects at several scales.

$$K_{eff} = K + (K - 1)(d - 1)$$

WORKED EXAMPLE
A 3 × 3 kernel always has 9 weights. With d = 2 it covers **5 × 5**; with d = 4 it covers **9 × 9**: a larger receptive field at no extra parameter cost and no loss of resolution.

6.5 Mask R-CNN

- **Mask R-CNN** (He et al., 2017) = Faster R-CNN + a small FCN **mask branch** that predicts a 28 × 28 binary mask for each detected region, in parallel with the class and box heads.
- **RoIAlign** samples features with bilinear interpolation instead of rounding coordinates (RoIPool), fixing misalignments that matter for pixel-accurate masks.
- Loss = $L_{cls} + L_{box} + L_{mask}$, with per-pixel binary cross-entropy for the mask of the predicted class only, so classes do not compete inside the mask.

6.6 Metrics and losses

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}, \qquad \text{Dice} = \frac{2\,|A \cap B|}{|A| + |B|} = \frac{2\,\text{IoU}}{1 + \text{IoU}}$$

WORKED EXAMPLE
Prediction 60 pixels, ground truth 50 pixels, overlap 40. IoU = 40 / (60 + 50 − 40) = 40/70 = **0.571**. Dice = 80 / 110 = **0.727**; check: 2·0.571 / 1.571 = 0.727. Dice is always ≥ IoU.
mIoU averages IoU over classes and is the standard semantic-segmentation metric.

- **Pixel accuracy** is misleading with imbalance: if 95% of pixels are background, predicting “all background” scores 95% accuracy but 0 IoU on the object.
- **Losses:** per-pixel cross-entropy (optionally class-weighted), **Dice loss** = 1 − Dice (robust to small foregrounds, common in medical imaging), or a sum of both.

7 Revision Kit

7.1 Choosing a model

Task	Good default	Output	Main metric
Image classification	ResNet-50, EfficientNet, ViT (fine-tuned)	Class probabilities	Top-1 / top-5 accuracy
Real-time detection	YOLO family (n/s for edge)	Boxes + classes + scores	mAP, FPS / latency
High-accuracy detection	Faster R-CNN + FPN, DETR variants	Boxes + classes + scores	COCO mAP@[.5:.95]
Semantic segmentation	U-Net (small data), DeepLabv3+	Class per pixel	mIoU, Dice
Instance segmentation	Mask R-CNN, YOLOv8-seg	Mask per object	Mask AP

7.2 One-page formula sheet

Concept	Formula / rule
Grayscale	$Y = 0.299R + 0.587G + 0.114B$
Normalization	$x' = (x - \mu_c) / \sigma_c$
Conv output size	$O = \lfloor (W - K + 2P) / S \rfloor + 1$
Conv parameters	$(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$
Conv compute	$K^2 \cdot C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out}$ MACs
Receptive field	n stacked 3 × 3 (stride 1) → (2n + 1) × (2n + 1)
Dilated kernel	$K_{eff} = K + (K - 1)(d - 1)$
Depthwise separable	$K^2 \cdot C_{in} + C_{in} \cdot C_{out}$
Residual block	$y = F(x) + x$; $\partial y / \partial x = \partial F / \partial x + I$
IoU	$\lvert A \cap B \rvert / (\lvert A \rvert + \lvert B \rvert - \lvert A \cap B \rvert)$
Precision / recall	TP / (TP + FP); TP / (TP + FN)
Focal loss	$-\alpha_t (1 - p_t)^{\gamma} \log p_t$
YOLO output	S × S × (B·5 + C)
YOLO decoding	$b_x = \sigma(t_x) + c_x$; $b_w = p_w e^{t_w}$
Transposed conv	$O = (I - 1)S - 2P + K$
Dice	$2\lvert A \cap B \rvert / (\lvert A \rvert + \lvert B \rvert) = 2IoU / (1 + IoU)$

7.3 Classic mistakes

- Mixing up BGR and RGB, or skipping the normalization the model was trained with.
- Forgetting padding in the output-size formula, or forgetting the bias in parameter counts.
- Thinking parameter count depends on image size (it does not; compute does).
- Calling degradation in plain deep nets “overfitting”; training error gets worse too.
- Assuming a residual shortcut always has zero parameters (projection shortcuts have weights).
- Computing IoU as intersection / (sum of areas) without subtracting the overlap.
- Running NMS across classes, or with a threshold so low that crowded objects disappear.
- Comparing mAP@0.5 with COCO mAP@[.5:.95].
- Forgetting that YOLOv1 predicts only one class per grid cell.
- Not undoing letterbox padding when mapping YOLO boxes back to the original image.
- Using pixel accuracy on imbalanced segmentation data.
- Applying geometric augmentation to images but not to boxes and masks.
- Leaving BatchNorm in training mode at inference.

7.4 Self-check before the MCQ assessment

- Can you apply a 3 × 3 Sobel kernel to a small patch by hand?
- Can you compute the output size and parameter count of a conv layer?
- Can you explain why residual connections fix the degradation problem, and count ResNet-50’s layers?
- Can you compute IoU for two boxes and run NMS on a short list?
- Can you derive YOLO’s output tensor and decode one predicted box into pixels?
- Can you choose between U-Net, DeepLab and Mask R-CNN, and compute IoU and Dice for a mask?

KEY IDEA

Next step: attempt the **Module 04 MCQ Assessment**. For each miss, return to the matching section, redo the worked example by hand and write one sentence on why the wrong option was tempting.